

## Cambridge (CIE) IGCSE Computer Science Paper 2: Algorithms, Programming and Logic (Set C)

**Tuesday 6 May 2025**

**Morning (Time: 1 hour 45 minutes)**

Total marks

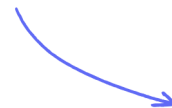
**/ 75**

### Instructions

- Try to complete this mock exam paper in one sitting, under exam conditions. Use all the time available and check your answers to each question at the end before submitting.
- Remember this is PRACTICE. Mistakes are fine and will help you improve in time for the real exam - just do your best.
- The total mark for this paper is 75.
- The number of marks for each question or part question is shown in brackets [ ].
- No marks will be awarded for using brand names of software packages or hardware.
- Calculators must not be used in this paper.

**Scan here to mark your mock exam**

or visit the mock exams landing page for this course



1 (a) Four pseudocode descriptions and five pseudocode statements are shown.

**Draw a line** to link each pseudocode description to the most appropriate pseudocode statement.

Some pseudocode statements will **not** be used

| Pseudocode description                                             | Pseudocode statement            |
|--------------------------------------------------------------------|---------------------------------|
| a loop that will always iterate at least once                      | FOR...TO...NEXT                 |
| a conditional statement to deal with many possible outcomes        | IF...THEN...ELSE...ENDIF        |
| a loop that will always iterate a set number of times              | WHILE...DO...ENDWHILE           |
| a conditional statement with different outcomes for true and false | CASE...OF...OTHERWISE...ENDCASE |
|                                                                    | REPEAT...UNTIL                  |

.....

.....

.....

.....

(4 marks)

(b) Using a single loop, **write an algorithm in pseudocode** to output 50 names that have been stored in the array, `Name[]`

.....

.....

---

(3 marks)

- (c) The variables P and Q are used to store data in a program. P stores a string. Q stores a character.

**Write pseudocode statements** to declare the variables P and Q, store "The world" in P and store 'W' in Q

---

---

(2 marks)

- 2 **Four** descriptions of stages in the program development life cycle are shown.

**Draw one line** to link each description to its most appropriate program development life cycle stage.

**Not all** program development life cycle stages will be used.

**Program development life cycle description**

|                                                                                                 |
|-------------------------------------------------------------------------------------------------|
| develop an algorithm to solve the problem by using structure diagrams, flowcharts or pseudocode |
| detect and fix the errors in the program                                                        |
| identify the problem and its requirements                                                       |
| write and implement the instructions to solve the problem                                       |

**Program development life cycle stage**

|            |
|------------|
| analysis   |
| coding     |
| design     |
| evaluation |
| testing    |

---

---

---

---

(4 marks)

**3 (a)** The variables X, Y and Z are used to store data in a program:

- X stores a string
- Y stores a position in the string (e.g. 2)
- Z stores the number of characters in the string.

The function **LENGTH(X)** finds the length of a string X.

The function **SUBSTRING(X,Y,Z)** finds a substring of X starting at position Y and Z characters long. The first character in X is in position 1.

**Write pseudocode statements to:**

- **store** the string "Programming is fun" in X
- **find the length** of the string and output it
- **extract the word** fun from the string and output it.

---

---

---

---

---

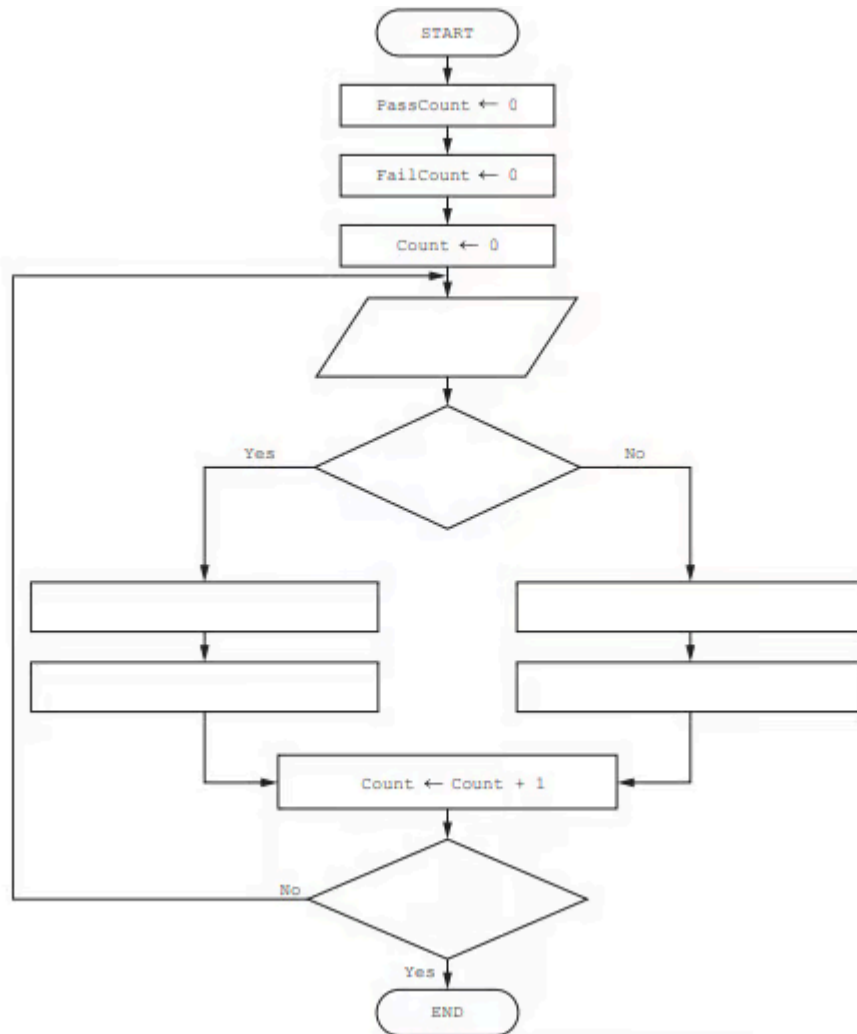
---

**(6 marks)**

**(b)** The flowchart shows an algorithm that should allow 60 test results to be entered into the variable **Score**.

Each test result is checked to see if it is 50 or more. If it is, the test result is assigned to the **Pass** array. Otherwise, it is assigned to the **Fail** array.

Complete this flowchart




---

---

---

---

---

---

---

---

(6 marks)

(c) Write a pseudocode routine that will check that each test result entered into the algorithm is between 0 and 100 inclusive.

.....

.....

.....

.....

**(4 marks)**

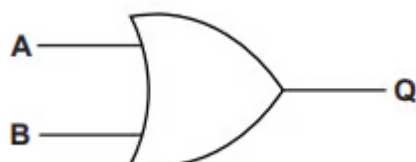
4 (a) **Complete** the truth table in **Fig. 1** for the Boolean statement  $P = \text{NOT}(A \text{ AND } B)$ .

| A | B | P |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 |   |
| 1 | 0 |   |
| 1 | 1 | 0 |

**Fig. 1**

(2 marks)

(b) **Complete** the truth table for the following logic gate.



| A | B | Q |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
|   | 0 |   |
| 1 |   |   |

---

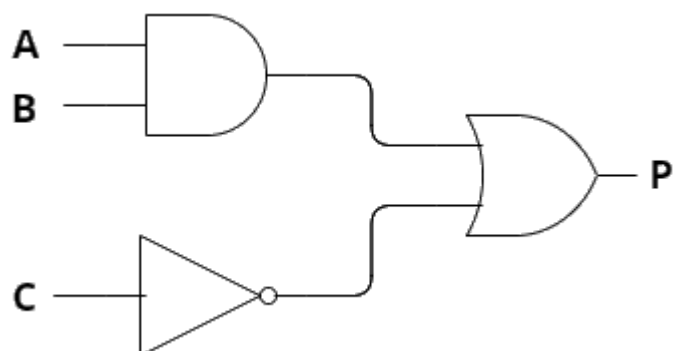
---

---

---

(4 marks)

(c) Write the **Boolean expression** represented by the logic diagram below:




---

---

---

(3 marks)

**5 (a) Describe** the purpose of validation and verification checks during data entry.

Include an **example** for each.

---

---

---

---

**(4 marks)**

**(b)** A range check is used to check that a value input is above 10 and below 20.

**Tick (✓) one box** to show which value would be accepted.

**A.** 10

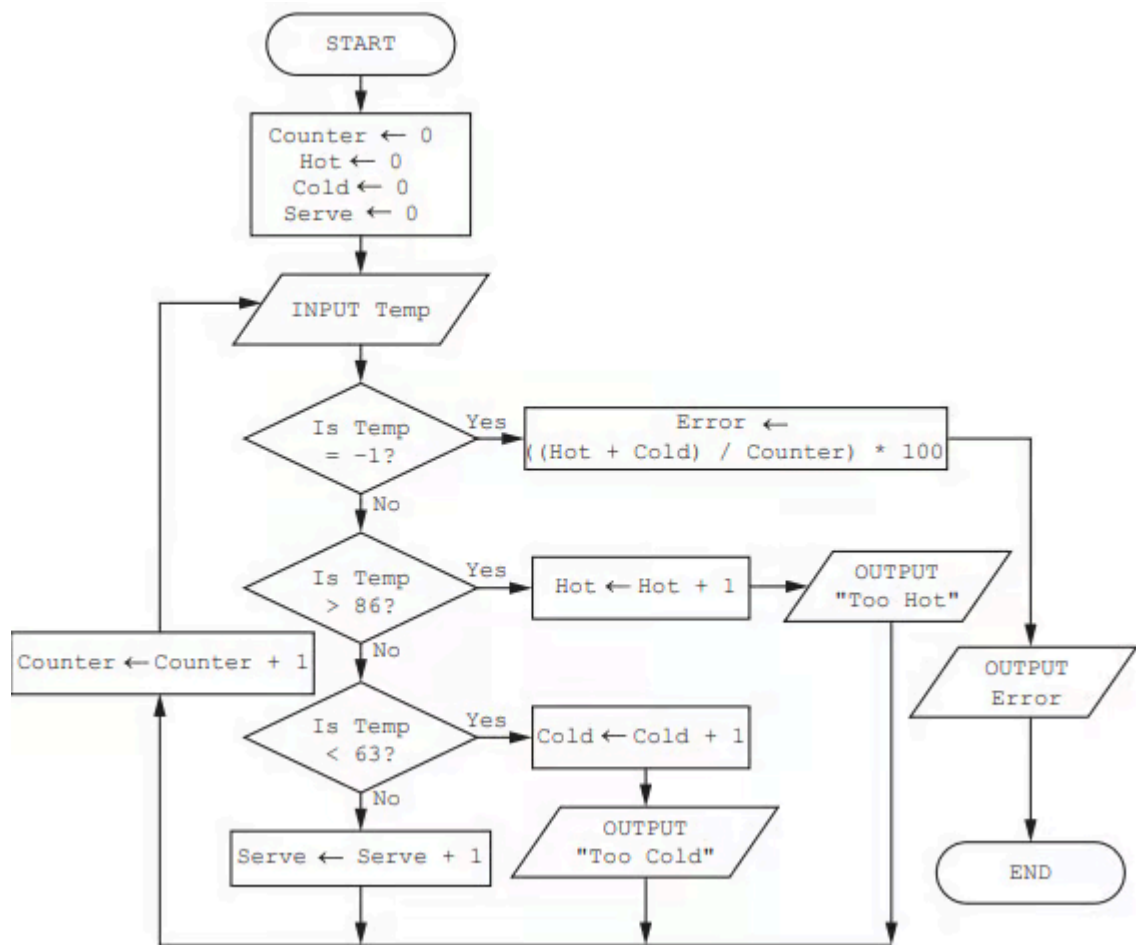
**B.** 15

**C.** 20

**D.** 30

**(1 mark)**

(c) This algorithm checks the temperature of hot food being served to customers.



Complete the trace table for the algorithm using this input data:

75, 78, 84, 87, 91, 80, 75, 70, 65, 62, -1, 20

| Counter | Hot | Cold | Serve | Temp | Error | OUTPUT |
|---------|-----|------|-------|------|-------|--------|
| -       | -   | -    | -     | -    | -     | -      |
| -       | -   | -    | -     | -    | -     | -      |

.....

.....

.....

.....

.....

.....

**(7 marks)**

- 6 (a) A database table, **PERFORMANCE**, is used to keep a record of the performances at a local theatre.

| ShowNumber | Type      | Title              | Date    | SoldOut |
|------------|-----------|--------------------|---------|---------|
| SN091      | Comedy    | An Evening at Home | 01 Sept | Yes     |
| SN102      | Drama     | Old Places         | 02 Oct  | No      |
| SN113      | Jazz      | Acoustic Evening   | 03 Nov  | No      |
| SN124      | Classical | Mozart Evening     | 04 Dec  | Yes     |
| SN021      | Classical | Bach Favourites    | 01 Feb  | Yes     |
| SN032      | Jazz      | 30 Years of Jazz   | 02 Mar  | Yes     |
| SN043      | Comedy    | Street Night       | 03 Apr  | No      |
| SN054      | Comedy    | Hoot               | 04 May  | No      |

**Show the output** that would be given by this structured query language (SQL) statement:

SELECT Date, Title

FROM PERFORMANCE

WHERE NOT SoldOut AND Type = "Jazz";

(2 marks)

(b) A database table, **2018MOV**, is used to keep a record of movie details.

| CatNo | Title                         | Genre1    | Genre2   | Blu-ray | DVD | Streaming |
|-------|-------------------------------|-----------|----------|---------|-----|-----------|
| 18m01 | Power Rangers                 | Adventure | Fantasy  | Yes     | No  | Yes       |
| 18m02 | Baywatch                      | Comedy    | Drama    | Yes     | No  | Yes       |
| 18m03 | Table 19                      | Comedy    | Drama    | Yes     | Yes | No        |
| 18m04 | Wonder Woman                  | Action    | Fantasy  | Yes     | No  | Yes       |
| 18m05 | Justice League                | Action    | Fantasy  | Yes     | Yes | Yes       |
| 18m06 | Twilight                      | Thriller  | Action   | Yes     | Yes | No        |
| 18m07 | Ant Man                       | Action    | Fantasy  | No      | Yes | No        |
| 18m08 | Venice Beach                  | Action    | History  | No      | Yes | No        |
| 18m12 | Fast Five                     | Action    | Thriller | No      | Yes | No        |
| 18m15 | King Kong                     | Adventure | Fantasy  | No      | Yes | No        |
| 18m16 | Transformers: The Last Knight | Action    | Sci-Fi   | Yes     | Yes | Yes       |
| 18m17 | The Dark Tower                | Fantasy   | Sci-Fi   | Yes     | Yes | No        |
| 18m19 | Beauty and the Beast          | Fantasy   | Romance  | Yes     | Yes | Yes       |
| 18m21 | The Mummy                     | Action    | Fantasy  | No      | No  | Yes       |
| 18m22 | Star Wars: Episode VIII       | Sci-Fi    | Action   | Yes     | No  | Yes       |
| 18m23 | Guardians of the Galaxy       | Action    | Sci-Fi   | Yes     | Yes | Yes       |
| 18m26 | Thor                          | Action    | Sci-Fi   | No      | Yes | Yes       |
| 18m27 | Twilight                      | Fantasy   | Sci-Fi   | No      | No  | Yes       |
| 18m30 | Beneath                       | Action    | Fantasy  | Yes     | No  | No        |
| 18m31 | Despicable Me                 | Animation | Action   | Yes     | Yes | No        |

Complete the structured query language (SQL) to return the category number and title for all Comedy movies.

SELECT CatNo, Title

.....2018MOV

WHERE Genre1 = .....

(2 marks)

(c) The table **students** stores information about students and their exam results. The table has the following structure:

| Field Name | Data Type | Description                         |
|------------|-----------|-------------------------------------|
| StudentID  | NUMERIC   | Unique identifier for each student  |
| FirstName  | STRING    | First name of the student           |
| LastName   | STRING    | Last name of the student            |
| Class      | STRING    | Class of the student (e.g., '10A')  |
| Subject    | STRING    | Subject of the exam (e.g., 'Maths') |
| Score      | NUMERIC   | Score obtained in the exam (0-100)  |

**Write an SQL query** to find the total sum of scores obtained by students in each class. The results should be ordered by class, and the output should display the class and the total sum of scores

---

---

---

---

**(4 marks)**

**7 Explain** how indexing could be used to search for a value stored in a one-dimensional array.

---

---

**(2 marks)**

8 The names of patients are stored in the one-dimensional (1D) array `Patient[]` of type string. A separate two-dimensional (2D) array `Readings[]` stores the latest data recorded about each patient. The array **already** contains the readings taken by a nurse for each patient:

- temperature measured to one decimal place
- pulse rate, a whole number.

Temperature readings should be in the range 31.6 to 37.2 inclusive.

Pulse readings should be in the range 55.0 to 100.0 inclusive.

The hospital number given to the patient is used for the index on both arrays, this is a value between 1 and 1000 inclusive.

When the data for a patient is checked a warning is given if any of the readings are out of range.

If both readings are out of range, then a severe warning is given.

**Write a procedure**, using pseudocode or program code, that meets the following requirements:

- takes the hospital number as a parameter
- checks if the number is valid
- outputs an error message and exits the procedure if the number is not valid
- if the hospital number is valid:
  - **output the patient's name**
  - **output 'Normal readings' if both the readings are within range**
  - **output 'Warning' and the name of the reading e.g. 'Pulse' if one reading is out of range**
  - **output 'Severe warning' and the names of the two readings 'Pulse and temperature' if both readings are out of range**
  - **exits the procedure.**

You do **not** need to initialise the data in the arrays.

[illegible] Save My Exams